

Blind 75 — Fill-in-the-Blank Skeletons

For each problem: fill the blanks, then retype from scratch without looking. Never copy-paste. Always type.

Pattern 1 — HashMap / Set

Two Sum (LC 1)

```
def twoSum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = _____
        if complement in seen:
            return _____
        seen[____] = ____
```

Fill: `target - num` | `[seen[complement], i]` | `num, i`

Contains Duplicate (LC 217)

```
def containsDuplicate(nums):
    seen = ____
    for num in nums:
        if num in seen:
            return ____
        seen.__(num)
    return ____
```

Fill: `set()` | `True` | `add` | `False`

Valid Anagram (LC 242)

```
def isAnagram(s, t):
    if _____ : return False
    count = {}
    for c in s:
        count[c] = count.get(c, 0) + ____
    for c in t:
        count[c] = count.get(c, 0) - ____
        if count[c] < ____: return False
    return True
```

Fill: `len(s) != len(t)` | `1` | `1` | `0`

Group Anagrams (LC 49)

```

from collections import defaultdict
def groupAnagrams(strs):
    groups = defaultdict(____)
    for s in strs:
        key = _____
        groups[key].append(s)
    return list(groups.values())

```

Fill: list | tuple(sorted(s))

Longest Consecutive Sequence (LC 128)

```

def longestConsecutive(nums):
    num_set = ____ (nums)
    best = 0
    for n in num_set:
        if _____ not in num_set: # only start a sequence at its beginning
            length = 1
            while _____ in num_set:
                length += 1
            best = max(best, length)
    return best

```

Fill: set | n - 1 | n + length

Pattern 2 — Two Pointers

Valid Palindrome (LC 125)

```

def isPalindrome(s):
    s = [c.lower() for c in s if c.isalnum()]
    left, right = ____, _____
    while left < right:
        if s[left] != s[right]: return ____
        left += 1
        right -= 1
    return ____

```

Fill: 0 | len(s) - 1 | False | True

3Sum (LC 15)

```

def threeSum(nums):
    nums.sort()
    result = []
    for i, val in enumerate(nums):
        if i > 0 and nums[i] == nums[i-1]: ____ # skip duplicates

```

```

left, right = ___, _____
while left < right:
    total = val + nums[left] + nums[right]
    if total == 0:
        result.append([val, nums[left], nums[right]])
        while left < right and nums[left] == nums[left+1]: left += 1
        left += 1
    elif total < 0: ___
    else: ___
return result

```

Fill: continue | i+1 | len(nums)-1 | left += 1 | right -= 1

Container With Most Water (LC 11)

```

def maxArea(height):
    left, right = ___, _____
    result = 0
    while left < right:
        area = _____ * _____
        result = max(result, area)
        if height[left] < height[right]: ___
        else: ___
    return result

```

Fill: 0 | len(height)-1 | min(height[left], height[right]) | (right - left) | left += 1 | right - = 1

Pattern 3 — Sliding Window

Longest Substring Without Repeating Characters (LC 3)

```

def lengthOfLongestSubstring(s):
    left = 0
    window = {}
    result = 0
    for right, char in enumerate(s):
        window[char] = window.get(char, 0) + 1
        while _____:
            window[s[left]] -= 1
            if window[s[left]] == 0: del window[s[left]]
            left += 1
        result = max(result, _____)
    return result

```

Fill: window[char] > 1 | right - left + 1

Longest Repeating Character Replacement (LC 424)

```
def characterReplacement(s, k):
    left = 0
    window = {}
    max_freq = 0
    result = 0
    for right, char in enumerate(s):
        window[char] = window.get(char, 0) + 1
        max_freq = max(max_freq, _____)
        while _____ > k:          # chars to replace = window size - most frequent
            window[s[left]] -= 1
            left += 1
        result = max(result, right - left + 1)
    return result
```

Fill: `window[char] | (right - left + 1) - max_freq`

Minimum Window Substring (LC 76)

```
from collections import Counter
def minWindow(s, t):
    need = Counter(t)
    missing = len(t)
    left = 0
    result = ""
    for right, char in enumerate(s):
        if need.get(char, 0) > 0: missing -= 1
        need[char] = need.get(char, 0) - 1
        while missing == 0:
            window = s[left:right+1]
            if not result or len(window) < len(result): result = ____
            if need[s[left]] == 0: missing += 1
            need[s[left]] += 1
            left += 1
    return result
```

Fill: `window`

Pattern 4 — Prefix / Suffix

Product of Array Except Self (LC 238)

```
def productExceptSelf(nums):
    n = len(nums)
    output = [1] * n
    for i in range(1, n):
        output[i] = _____ # left product
    right = 1
    for i in range(n-2, -1, -1):
```

```
        right *= _____
        output[i] *= ____
    return output
```

Fill: `output[i-1] * nums[i-1]` | `nums[i+1]` | `right`

Maximum Subarray (LC 53) — Kadane's

```
def maxSubArray(nums):
    max_sum = curr = nums[0]
    for num in nums[1:]:
        curr = max(____, _____)
        max_sum = max(max_sum, curr)
    return max_sum
```

Fill: `num` | `curr + num`

Maximum Product Subarray (LC 152)

```
def maxProduct(nums):
    max_p = min_p = result = nums[0]
    for num in nums[1:]:
        candidates = (num, max_p * num, min_p * num)
        max_p = max(____)
        min_p = min(____)
        result = max(result, max_p)
    return result
```

Fill: `candidates` | `candidates`

Pattern 5 — Binary Search

Find Minimum in Rotated Sorted Array (LC 153)

```
def findMin(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[right]:
            left = ____
        else:
            right = ____
    return nums[left]
```

Fill: `mid + 1` | `mid`

Search in Rotated Sorted Array (LC 33)

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target: return mid
        if nums[left] <= nums[mid]:          # left half is sorted
            if nums[left] <= target < nums[mid]: right = mid - 1
            else: left = ____
        else:                               # right half is sorted
            if nums[mid] < target <= nums[right]: left = mid + 1
            else: right = ____
    return -1
```

Fill: mid + 1 | mid - 1

Pattern 6 — Dynamic Programming (1D)

Climbing Stairs (LC 70)

```
def climbStairs(n):
    if n <= 2: return n
    a, b = 1, 2
    for _ in range(3, n+1):
        a, b = ____, _____
    return b
```

Fill: b | a + b

House Robber (LC 198)

```
def rob(nums):
    prev2, prev1 = 0, 0
    for num in nums:
        prev2, prev1 = prev1, max(prev1, _____)
    return prev1
```

Fill: prev2 + num

House Robber II (LC 213)

```
def rob(nums):
    def rob_linear(arr):
        prev2, prev1 = 0, 0
        for num in arr:
            prev2, prev1 = prev1, max(prev1, prev2 + num)
        return prev1
```

```
if len(nums) == 1: return nums[0]
return max(rob_linear(___), rob_linear(___)) # exclude first OR exclude last
```

Fill: `nums[:-1]` | `nums[1:]`

Coin Change (LC 322)

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = __
    for coin in coins:
        for i in range(coin, amount + 1):
            dp[i] = min(dp[i], _____)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Fill: `0` | `dp[i - coin] + 1`

Jump Game (LC 55)

```
def canJump(nums):
    max_reach = 0
    for i, jump in enumerate(nums):
        if i > max_reach: return __
        max_reach = max(max_reach, _____)
    return __
```

Fill: `False` | `i + jump` | `True`

Longest Increasing Subsequence (LC 300)

```
def lengthOfLIS(nums):
    dp = [1] * len(nums)
    for i in range(1, len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], _____)
    return max(dp)
```

Fill: `dp[j] + 1`

Word Break (LC 139)

```
def wordBreak(s, wordDict):
    word_set = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = __
    for i in range(1, len(s) + 1):
```

```

        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = ____
                break
    return dp[len(s)]

```

Fill: True | True

Combination Sum IV (LC 377)

```

def combinationSum4(nums, target):
    dp = [0] * (target + 1)
    dp[0] = ____
    for i in range(1, target + 1):
        for num in nums:
            if i >= num:
                dp[i] += _____
    return dp[target]

```

Fill: 1 | dp[i - num]

Decode Ways (LC 91)

```

def numDecodings(s):
    if s[0] == '0': return 0
    dp = [0] * (len(s) + 1)
    dp[0], dp[1] = 1, 1
    for i in range(2, len(s) + 1):
        one = int(s[i-1])
        two = int(s[i-2:i])
        if one != 0: dp[i] += ____
        if 10 <= two <= 26: dp[i] += ____
    return dp[-1]

```

Fill: dp[i-1] | dp[i-2]

Pattern 7 — 2D Dynamic Programming

Unique Paths (LC 62)

```

def uniquePaths(m, n):
    dp = [[1] * n for _ in range(m)]
    for r in range(1, m):
        for c in range(1, n):
            dp[r][c] = _____ + _____
    return dp[m-1][n-1]

```


Fill: `dp[r-1][c] | dp[r][c-1]`

Longest Common Subsequence (LC 1143)

```
def longestCommonSubsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = _____ + 1
            else:
                dp[i][j] = max(_____, _____)
    return dp[m][n]
```

Fill: `dp[i-1][j-1] | dp[i-1][j] | dp[i][j-1]`

Pattern 8 — Interval

Merge Intervals (LC 56)

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    result = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])
    return result
```

Fill: `end`

Insert Interval (LC 57)

```
def insert(intervals, newInterval):
    result = []
    for i, (start, end) in enumerate(intervals):
        if newInterval[1] < start:
            result.append(newInterval)
            return result + intervals[i:]
        elif newInterval[0] > end:
            result.append([start, end])
        else:
            newInterval = [min(newInterval[0], start), max(newInterval[1], end)]
    result.append(newInterval)
    return result
```

Fill: newInterval

Non-overlapping Intervals (LC 435)

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by END
    count = 0
    end = float('-inf')
    for start, e in intervals:
        if start >= end:
            end = e
        else:
            count += 1
    return count
```

Fill: e | 1

Meeting Rooms II (LC 253)

```
import heapq
def minMeetingRooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []  # stores end times
    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heappop(heap)
        heapq.heappush(heap, end)
    return len(heap)
```

Fill: end

Pattern 9 — Graph BFS / DFS

Number of Islands (LC 200)

```
from collections import deque
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    visited = set()
    count = 0
    def bfs(r, c):
        q = deque([(r, c)])
        visited.add((r, c))
        while q:
            row, col = q.popleft()
            for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
                nr, nc = row+dr, col+dc
                if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited and grid[nr]
```

```

[nc]=="1":
    visited.add((nr, nc))
    q.append(____)
for r in range(rows):
    for c in range(cols):
        if grid[r][c] == "1" and (r,c) not in visited:
            bfs(r, c)
            count += ____
return count

```

Fill: (nr, nc) | 1

Course Schedule (LC 207) — Cycle detection

```

def canFinish(numCourses, prerequisites):
    graph = {i: [] for i in range(numCourses)}
    for a, b in prerequisites:
        graph[b].append(a)
    # 0=unvisited, 1=visiting, 2=done
    state = [0] * numCourses
    def dfs(node):
        if state[node] == 1: return ____ # cycle
        if state[node] == 2: return True
        state[node] = 1
        for nei in graph[node]:
            if not dfs(nei): return False
        state[node] = ____
        return True
    return all(dfs(i) for i in range(numCourses))

```

Fill: False | 2

Pacific Atlantic Water Flow (LC 417)

```

from collections import deque
def pacificAtlantic(heights):
    rows, cols = len(heights), len(heights[0])
    def bfs(starts):
        q = deque(starts)
        visited = set(starts)
        while q:
            r, c = q.popleft()
            for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited and heights[nr][nc]
                >= heights[r][c]:
                    visited.add((nr, nc))
                    q.append((nr, nc))
        return visited

```

```

pacific = bfs([(r,0) for r in range(rows)] + [(0,c) for c in range(cols)])
atlantic = bfs([(r,cols-1) for r in range(rows)] + [(rows-1,c) for c in range(cols)])
return list(_____) # cells reachable from both

```

Fill: pacific & atlantic

Clone Graph (LC 133)

```

def cloneGraph(node):
    if not node: return None
    clones = {}
    def dfs(n):
        if n in clones: return ____
        clones[n] = Node(n.val)
        for nei in n.neighbors:
            clones[n].neighbors.append(____)
        return clones[n]
    return dfs(node)

```

Fill: clones[n] | dfs(nei)

Pattern 10 — Linked List

Reverse Linked List (LC 206)

```

def reverseList(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = ____
        prev = ____
        curr = ____
    return prev

```

Fill: prev | curr | nxt

Detect Cycle (LC 141)

```

def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = ____
        fast = ____
        if slow == fast: return True
    return False

```

Fill: `slow.next` | `fast.next.next`

Merge Two Sorted Lists (LC 21)

```
def mergeTwoLists(l1, l2):
    dummy = ListNode(0)
    curr = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            curr.next = l1
            l1 = l1.next
        else:
            curr.next = l2
            l2 = l2.next
        curr = curr.next
    curr.next = l1 or l2
    return dummy.next
```

Fill: `l1.next` | `l2.next` | `curr.next`

Remove Nth Node From End (LC 19)

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0)
    dummy.next = head
    left = right = dummy
    for _ in range(n + 1): # advance right n+1 steps
        right = right.next
    while right:
        left = left.next
        right = right.next
    left.next = None
    return dummy.next
```

Fill: `right.next` | `right.next` | `left.next.next`

Reorder List (LC 143)

```
def reorderList(head):
    # Step 1: find middle
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    # Step 2: reverse second half
    prev, curr = None, slow.next
    slow.next = None
    while curr:
        nxt = curr.next
```

```

        curr.next = prev
        prev, curr = curr, nxt
# Step 3: merge two halves
l1, l2 = head, prev
while l2:
    l1.next, l2.next, l1, l2 = l2, ___, l1.next, l2.next

```

Fill: l1.next

Merge K Sorted Lists (LC 23)

```

import heapq
def mergeKLists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = curr = ListNode(0)
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next

```

Fill: node.next.val | node.next

Pattern 11 — Tree DFS

Maximum Depth of Binary Tree (LC 104)

```

def maxDepth(root):
    if not root: return __
    return 1 + max(_____, _____)

```

Fill: 0 | maxDepth(root.left) | maxDepth(root.right)

Same Tree (LC 100)

```

def isSameTree(p, q):
    if not p and not q: return __
    if not p or not q: return __
    if p.val != q.val: return __
    return isSameTree(____, ____ and isSameTree(____, ____

```

Fill: True | False | False | p.left, q.left | p.right, q.right

Invert Binary Tree (LC 226)

```
def invertTree(root):  
    if not root: return None  
    root.left, root.right = ____, ___  
    invertTree(root.left)  
    invertTree(root.right)  
    return root
```

Fill: `invertTree(root.right)` | `invertTree(root.left)`

Validate BST (LC 98)

```
def isValidBST(root):  
    def validate(node, low, high):  
        if not node: return ____  
        if not (____ < node.val < ____): return False  
        return validate(node.left, low, node.val) and validate(node.right, node.val, high)  
    return validate(root, float('-inf'), float('inf'))
```

Fill: `True` | `low` | `high`

Kth Smallest in BST (LC 230)

```
def kthSmallest(root, k):  
    stack = []  
    curr = root  
    while stack or curr:  
        while curr:  
            stack.append(curr)  
            curr = ____ # go left  
        curr = stack.pop()  
        k -= 1  
        if k == 0: return ____  
        curr = ____ # go right
```

Fill: `curr.left` | `curr.val` | `curr.right`

Lowest Common Ancestor of BST (LC 235)

```
def lowestCommonAncestor(root, p, q):  
    curr = root  
    while curr:  
        if p.val < curr.val and q.val < curr.val:  
            curr = ____  
        elif p.val > curr.val and q.val > curr.val:  
            curr = ____
```

```
else:
    return ____
```

Fill: `curr.left` | `curr.right` | `curr`

Binary Tree Maximum Path Sum (LC 124)

```
def maxPathSum(root):
    result = [root.val]
    def dfs(node):
        if not node: return 0
        left = max(____, 0) # ignore negative paths
        right = max(____, 0)
        result[0] = max(result[0], node.val + left + right)
        return node.val + max(left, right)
    dfs(root)
    return result[0]
```

Fill: `dfs(node.left)` | `dfs(node.right)`

Subtree of Another Tree (LC 572)

```
def isSubtree(root, subRoot):
    if not root: return ____
    if isSameTree(root, subRoot): return ____
    return isSubtree(____, subRoot) or isSubtree(____, subRoot)

def isSameTree(p, q):
    if not p and not q: return True
    if not p or not q: return False
    return p.val == q.val and isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

Fill: `False` | `True` | `root.left` | `root.right`

Pattern 12 — Tree BFS

Binary Tree Level Order Traversal (LC 102)

```
from collections import deque
def levelOrder(root):
    if not root: return []
    q = deque([root])
    result = []
    while q:
        level = []
        for _ in range(____): # process one level at a time
            node = q.popleft()
            level.append(node.val)
```



```

        if node.left: q.append(____)
        if node.right: q.append(____)
    result.append(level)
return result

```

Fill: `len(q)` | `node.left` | `node.right`

Pattern 13 — Heap

Top K Frequent Elements (LC 347)

```

import heapq
def topKFrequent(nums, k):
    count = {}
    for n in nums:
        count[n] = count.get(n, 0) + 1
    heap = []
    for num, freq in count.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            ____
    return [num for freq, num in heap]

```

Fill: `heapq.heappop(heap)`

Find Median from Data Stream (LC 295)

```

import heapq
class MedianFinder:
    def __init__(self):
        self.small = [] # max-heap (negate values)
        self.large = [] # min-heap

    def addNum(self, num):
        heapq.heappush(self.small, ____ ) # always push to small first
        if self.small and self.large and -self.small[0] > self.large[0]:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return ____
        return (____ + self.large[0]) / 2

```

Fill: `-num` | `-self.small[0]` | `-self.small[0]`

Pattern 14 — Stack

Valid Parentheses (LC 20)

```
def isValid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for c in s:
        if c in pairs:
            if not stack or stack[-1] != pairs[c]: return ____
            stack.pop()
        else:
            stack.append(____)
    return len(stack) == ____
```

Fill: False | c | 0

Pattern 15 — Trie

Implement Trie (LC 208)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = ____

    def insert(self, word):
        curr = self.root
        for c in word:
            if c not in curr.children:
                curr.children[c] = ____
            curr = curr.children[c]
        curr.is_end = ____

    def search(self, word):
        curr = self.root
        for c in word:
            if c not in curr.children: return False
            curr = curr.children[c]
        return ____

    def startsWith(self, prefix):
        curr = self.root
        for c in prefix:
            if c not in curr.children: return False
```

```
curr = curr.children[c]
return ____
```

Fill: TrieNode() | TrieNode() | True | curr.is_end | True

Pattern 16 — Matrix

Spiral Matrix (LC 54)

```
def spiralOrder(matrix):
    result = []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1): result.append(matrix[top][c]); top += 1
        for r in range(top, bottom + 1): result.append(matrix[r][right]); right -= 1
        if top <= bottom:
            for c in range(right, left-1, -1): result.append(matrix[bottom][c]); bottom -= 1
        if left <= right:
            for r in range(bottom, top-1, -1): result.append(matrix[r][left]); left += 1
    return result
```

Blank: guards `if top <= bottom` and `if left <= right` before 3rd and 4th moves.

Rotate Image (LC 48)

```
def rotate(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i+1, n):
            matrix[i][j], matrix[j][i] = ____, __
    for row in matrix:
        row.reverse()
```

Fill: `matrix[j][i]` | `matrix[i][j]`

Word Search (LC 79)

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    visited = set()
    def dfs(r, c, i):
        if i == len(word): return ____
        if not(0<=r<rows and 0<=c<cols) or board[r][c] != word[i] or (r,c) in visited:
            return False
        visited.add((r, c))
        res = (dfs(r+1,c,i+1) or dfs(r-1,c,i+1) or dfs(r,c+1,i+1) or dfs(r,c-1,i+1))
        visited.remove(____)
```

```

        return res
    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0): return True
    return False

```

Fill: True | (r, c)

Pattern 17 — Bit Manipulation

Number of 1 Bits (LC 191)

```

def hammingWeight(n):
    count = 0
    while n:
        count += n & ____
        n >>= ____
    return count

```

Fill: 1 | 1

Counting Bits (LC 338)

```

def countBits(n):
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[____] + ____ # offset = i & (i-1) clears lowest bit
    return dp

```

Fill: i >> 1 | i & 1

Missing Number (LC 268)

```

def missingNumber(nums):
    n = len(nums)
    return _____ - sum(nums) # expected sum - actual sum

```

Fill: n * (n + 1) // 2

Sum of Two Integers (LC 371)

```

def getSum(a, b):
    mask = 0xFFFFFFFF
    while b & mask:
        carry = (a & b) << 1
        a = a ^ b

```

```
    b = ____
    return a if b == 0 else ____
```

Fill: carry | a & 0xFFFFFFFF (handles negative overflow in Python)

Pattern 18 — String

Longest Palindromic Substring (LC 5)

```
def longestPalindrome(s):
    result = ""
    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left+1:right]    # slice AFTER loop exits
    for i in range(len(s)):
        odd = expand(i, i)        # odd length
        even = expand(i, ____)    # even length
        for p in [odd, even]:
            if len(p) > len(result): result = p
    return result
```

Fill: i + 1

Encode and Decode Strings (LC 271)

```
def encode(strs):
    return "".join(f"{len(s)}#{s}" for s in strs)

def decode(s):
    result = []
    i = 0
    while i < len(s):
        j = s.index('#', i)
        length = int(s[i:j])
        result.append(s[j+1 : ____])
        i = ____
    return result
```

Fill: j + 1 + length | j + 1 + length

How to use this file

1. Pick a problem. Cover the Fill line.
2. Try to fill the blanks from memory.
3. Check. Then close the file and retype the entire solution from scratch.
4. Repeat the same problem next day — cold recall before anything else.